T1 配对（pairs）

固定右端点，查询合格左端点的前缀数量

条件 $a_i < i < a_j < j$ 可以拆为：右端点满足 $a_j < j$；左端点满足 $a_i < i$，并且编号小于 a_j。一个位置能否作为左端点，只取决于它自己的标签和编号，适合先做前缀计数。

令 $s[x]$ 为前 x 个位置中满足 $a_i < i$ 的位置数。取 $s[0] = 0$，依次计算

$$s[x] = s[x-1] + [a_x < x],$$

方括号在条件成立时为 1，否则为 0。

固定右端点 j。若 $0 < a_j < j$，合法左端点恰好是前 $a_j - 1$ 个位置中的合格位置，贡献为 $s[a_j - 1]$；若 $a_j = 0$ 或 $a_j \geq j$，贡献为 0。前者不存在编号小于 0 的左端点，后者违反右端点自身的严格不等式。

这样计入的位置均满足原来的三个比较；反之，每个合法配对都会在处理其唯一的右端点时被计入一次。先判断范围再查询，也避免了负下标以及把可能达到 10^9 的标签直接用作下标。

每组扫描两遍，时间 $O(n)$、空间 $O(n)$；所有组总时间 $O(\sum n)$。答案至多 $n(n-1)/2$，可能超过 32 位整数，使用 long long。每组覆盖前缀数组并清零答案。

满分参考代码

```cpp
#include<bits/stdc++.h>
using namespace std;
const int N = 2e5 + 10;
int n,t,a[N],s[N];
int main(){
    freopen("pairs.in","r",stdin);
    freopen("pairs.out","w",stdout);
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin >> t;
    while(t--){
        cin >> n;
        s[0] = 0;
        for(int i = 1; i <= n; i++){
            cin >> a[i];
            s[i] = s[i - 1];
            if(a[i] < i) s[i]++;
        }
        long long ans = 0;
        for(int j = 1; j <= n; j++){
            if(a[j] > 0 && a[j] < j){
                ans += s[a[j] - 1];
            }
        }
        cout << ans << '\n';
    }
    return 0;
}
```

部分分

每点 5 分；第 2 项已覆盖第 1 项的点组，保证分不再相加。

序号	条件	点号与保证分	做法与复杂度
1	每组 $n \le 1500$，$\sum n \le 20000$	$1 \sim 6$，30 分	枚举 $i < j$，直接检查 $a_i < i < a_j < j$。单组时间 $O(n^2)$，空间 $O(n)$；全文件时间 $O(\sum n^2)$。
2	上一档或 $a_i \le 300$	$1 \sim 12$，60 分	先跳过 $a_j \ge j$，仅枚举 $1 \le i < a_j$，检查 $a_i < i$；$a_j = 0$ 时没有候选。设最大标签为 V，单组时间 $O(n(1 + \min(n, V)))$，空间 $O(n)$。
—	完整范围	$13 \sim 20$，40 分	用前缀计数回答每个右端点的贡献，单组时间、空间 $O(n)$。

T2 通行（rail）

差分统计次数，每段路独立选收费方式

路线已经给定，只需统计每段路经过几次，再决定是否为它买卡。若某段通过 x 次，买卡后每次都比原价便宜，所以把购卡时间提前到第一次通过前不会更贵。该路段只需比较

$$xA \quad 与 \quad C + xB.$$

不同路段没有共享预算或购卡数量限制，各自较便宜的方式可以同时采用；因此把各段的最小费用相加，就得到全局最小值。$x = 0$ 时自然选择费用为 0 的不买卡方案。

一次从车站 u 到 v 的移动，令 $l = \min(u, v)$、$r = \max(u, v)$，经过的路段恰好是 $[l, r - 1]$。最终只需要各路段的经过次数，可以用差分数组记录整段加一：

$$d_l \mathrel{+}= 1, \qquad d_r \mathrel{-}= 1.$$

这里在 r 处减一，因为第 r 段路已不在此次行程中。从左向右求前缀和 cnt += d[i] 时，每次移动恰好给 $l \le i < r$ 的路段贡献一次，故得到准确次数。随后立即将 $\min(\text{cnt}\, A_i, C_i + \text{cnt}\, B_i)$ 加入答案即可。

共有 $m - 1$ 次区间更新和 $n - 1$ 次路段计算，总时间 $O(n + m)$；代码保存行程与差分数组，空间 $O(n + m)$。d[n] 可以作为结束标记，但没有第 n 段路。费用乘积可能超过 10^9，总代价小于 10^{15}，乘法和累加均用 long long。

满分参考代码

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 100000 + 5;

int n,m,p[N];
long long d[N];

int main(){
    freopen("rail.in","r",stdin);
    freopen("rail.out","w",stdout);
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin >> n >> m;
    for(int i = 1; i <= m; i++) cin >> p[i];
    for(int i = 1; i < m; i++){
        int l = min(p[i],p[i + 1]);
        int r = max(p[i],p[i + 1]);
        d[l]++;
        d[r]--;
    }
    long long cnt = 0,ans = 0;
    for(int i = 1; i < n; i++){
        long long a,b,c;
        cin >> a >> b >> c;
        cnt += d[i];
        ans += min(cnt * a,c + cnt * b);
    }
    cout << ans << '\n';
    return 0;
}
```

部分分

每点 5 分；第 2 项保证覆盖前 10 点，共 50 分。

序号	条件	点号与保证分	做法与复杂度
1	$n \le 18, m \le 200$	$1 \sim 4$，20 分	逐次移动、逐路段统计经过次数，再枚举每段买卡或不买卡，累加费用并取最小。提前购卡的理由沿用正文。时间 $O(nm + 2^n)$，空间 $O(n + m)$。
2	$n, m \le 1500$	$1 \sim 10$，50 分	仍逐路段累加经过次数，但每段直接取两种费用较小者。时间 $O(n + m + \sum \lvert p_{i+1} - p_i \rvert)$，上界 $O(nm)$；空间 $O(n + m)$。
—	完整范围	$11 \sim 20$，50 分	差分恢复次数，再逐段计算最小费用，时间和空间均为 $O(n + m)$。

T3 远行（visit）

保留最远者，其余名额选访问积分最高的驿站

所有驿站在入口同一侧。选定一个非空集合后，最短路线是走到最远的驿站再返回，所得积分为

$$\sum a_i + 2\max s_i.$$

行走积分只需一名最远驿站负责，其余名额应尽量提高访问积分。为此把所有驿站按 a 降序排列，距离随之移动；以下下标都指排序后的编号，距离不再有序。

固定要选的数量 k。取任意最优方案，保留其中一名最远驿站 j：

- 若 $j \le k$，改选前 k 名。访问积分不会减少，且仍保留 j，行走积分也不会减少。
- 若 $j > k$，保留 j，其余名额改选前 $k-1$ 名。这些位置与 j 不同，访问积分之和最大；仍保留原最远者，行走积分不会减少。

所以只需考虑“前 k 名”，以及“前 $k-1$ 名加一名 $j > k$ 的驿站”。

令 $pre[k]$ 为前 k 名访问积分之和，$far[k]$ 为其中最大距离。第一类候选为

$$B_k = pre[k] + 2far[k].$$

第二类的真实积分为 $pre[k-1] + a_j + 2\max(far[k-1], s_j)$。若 $s_j \le far[k-1]$，它没有增加路程，且 $a_j \le a_k$，不可能优于 B_k；若 $s_j > far[k-1]$，则只需最大化 $a_j + 2s_j$。

因此从右向左预处理

$$suf[i] = \max_{j \ge i}(a_j + 2s_j),$$

就能同时求出所有答案：

$$ans_k = \max\big(B_k,\ pre[k-1] + suf[k+1]\big) \quad (k < n).$$

后缀中不够远的驿站也参与了最大值，但其计算值不超过真实积分，而真实积分又不超过 B_k，不会抬高答案。所有可能改进第一项的方案则都被第二项准确计算。$k = n$ 时没有额外驿站，只取 B_n。

取 $pre[0] = far[0] = 0$，从左向右求前缀和及最远距离，再从右向左求后缀最大值。距离非负、访问积分为正，这些初值与空前缀一致。排序耗时 $O(n\log n)$，其余扫描 $O(n)$，总时间 $O(n\log n)$、空间 $O(n)$；积分使用 long long。

满分参考代码

```cpp
#include<bits/stdc++.h>
using namespace std;
const int N = 1e5 + 10;
struct node{
    int s,a;
}b[N];
int n,far[N];
long long pre[N],suf[N];
bool cmp(node x,node y){
    return x.a > y.a;
}
int main(){
    freopen("visit.in","r",stdin);
    freopen("visit.out","w",stdout);
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> b[i].s;
    for(int i = 1; i <= n; i++) cin >> b[i].a;
    sort(b + 1,b + n + 1,cmp);
    for(int i = 1; i <= n; i++){
        pre[i] = pre[i - 1] + b[i].a;
        far[i] = max(far[i - 1],b[i].s);
    }
    for(int i = n; i >= 1; i--){
        suf[i] = max(suf[i + 1],b[i].a + 2LL * b[i].s);
    }
    for(int k = 1; k <= n; k++){
        long long ans = pre[k] + 2LL * far[k];
        if(k < n) ans = max(ans,pre[k - 1] + suf[k + 1]);
        cout << ans << '\n';
    }
    return 0;
}
```

部分分

每点 5 分，保证分按公开点号计算；某种二次实现额外通过一般点，不改变其保证范围。

序号	条件	点号与保证分	做法与复杂度
1	$n \le 18$	$1 \sim 4$，20 分	搜索各驿站选或不选，维护数量、访问积分和最远距离，在叶子更新对应数量的答案。每步 $O(1)$，总时间 $O(2^n)$，空间 $O(n)$。
2	$n \le 1500$	$1 \sim 8$，40 分	沿原来的距离顺序固定最大选中编号 r，必须选它；其余从此前驿站中按访问积分从高到低加入，依次更新所有数量的答案。维护有序数组，每轮先更新答案，再插入 a_r，避免重复选择。时间 $O(n^2)$，空间 $O(n)$。
—	完整范围	$9 \sim 20$，60 分	按访问积分排序，用前缀和、最远距离及后缀最大值同时计算所有 k，时间 $O(n \log n)$，空间 $O(n)$。

T4 丰收（harvest）

看最后一块田是否被收割

每段长度固定为 m，选中以 i 为右端点的一段，其产量就是前缀和之差 $s_i - s_{i-m}$。要衔接多段，还需要知道已经使用的田地范围，以及已选段数；具体怎样选到当前最优产量，不会影响后续可用位置。

于是令 $f[j][i]$ 表示只用前 i 块田，恰好收割 j 段的最大产量。考察第 i 块是否收割：

- 不收割：全部 j 段都在前 $i-1$ 块中，值为 $f[j][i-1]$。
- 收割：包含它的最后一段只能是 $[i-m+1, i]$；前 $j-1$ 段必须位于前 $i-m$ 块中，值为 $f[j-1][i-m] + s_i - s_{i-m}$。

两种情况覆盖所有方案；接上的新段与前缀不重叠，相邻也允许。因此

$$f[j][i] = \max\big(f[j][i-1],\ f[j-1][i-m] + s_i - s_{i-m}\big) \qquad (i \geq jm).$$

不选任何段时 $f[0][i] = 0$。当 $j > 0$ 且 $i < jm$ 时，位置不足，记为负无穷；不能用 0 代替不可行状态。代码按 j 递增计算各层，同一层按 i 递增，两个来源均已求好。转移只处理 $i \geq jm$，所读上一层也能放下 $j-1$ 段。答案是 $f[k][n]$，满足题目要求的恰好 k 段。

最多计算 $nk \leq 25000000$ 个状态，每个只比较两个值，时间 $O(nk)$。保留完整 DP 表，空间 $O(nk)$；代码的 5010×5010 个 long long 约占 191.5 MiB，符合 512 MiB 限制。总产量最多 5×10^{12}，前缀和和 DP 均用 long long。

满分参考代码

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 5010;
const long long INF = 4e18;
int n,m,k;
long long a[N],s[N],f[N][N];

int main(){
    freopen("harvest.in","r",stdin);
    freopen("harvest.out","w",stdout);
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin >> n >> m >> k;
    for(int i = 1; i <= n; i++){
        cin >> a[i];
        s[i] = s[i - 1] + a[i];
    }
    // f[j][i]：前 i 块田中，恰好收割 j 段的最大产量。
    for(int j = 1; j <= k; j++){
        for(int i = 0; i <= n; i++){
            f[j][i] = -INF;
        }
        for(int i = j * m; i <= n; i++){
            f[j][i] = f[j][i - 1];
            long long x = f[j - 1][i - m] + s[i] - s[i - m];
            if(x > f[j][i]) f[j][i] = x;
        }
    }
    cout << f[k][n] << '\n';
    return 0;
}
```

部分分

每点 5 分；第 2 项覆盖前两组，保证 40 分。

序号	条件	点号与保证分	做法与复杂度
1	$n \le 16$	$1 \sim 4$，20 分	从左到右枚举每段的右端点，下一端点至少比上一端点大 m，并为未选段留足位置；选满 k 段后更新答案。叶子数为 $\binom{n-km+k}{k}$，时间上界 $O(n + k\binom{n-km+k}{k})$，空间 $O(n)$。
2	$n \le 180$	$1 \sim 8$，40 分	令 $g[j][i]$ 为第 j 段恰好在 i 结束的最大产量。枚举上一段端点 $(j-1)m \le p \le i-m$，取 $g[j-1][p] + s_i - s_{i-m}$ 的最大值。首层是一段的产量，不可达为负无穷，答案取最后一层最大值。时间 $O(kn^2)$，空间 $O(nk)$。
—	完整范围	$9 \sim 20$，60 分	用正文的前缀最优状态直接比较“跳过、取最后一段”，时间、空间均为 $O(nk)$。

第一项把每个已选长度为 m 的段压成一个标记后，剩余 $n - km$ 个空位置与 k 个标记共长 $n - km + k$，因此完整方案数为上述组合数；每个搜索结点都能延伸至合法叶子。